

Course Code: CS1083

Name of the Programme: BCA Computer Science (Hons.)

Name of the Course: Data Structures using C

Semester: II

Course credit	No. of hours per week (L + T + P)	Total no. of Teaching hours
4	2+0+4	90
Course Objectives:		
a) Understand fundamental programming concepts essential for implementing data structures.		
b) Apply user-defined data types to implement linear data structures like linked lists, stacks, and queues.		
c) Implement binary trees and perform various traversal methods including inorder, preorder, and postorder.		
d) Analyze and apply tree-based data structures for efficient data storage and retrieval.		

Course outline (Syllabus of the course)

Syllabus:

Module - 1	No. of hours
	6
Foundations of Programming for data structures: Arrays, Pointers and Strings Functions and Recursion, Arrays, Pointers and Strings: Introduction to Arrays, Definition, One Dimensional Array and Multidimensional Arrays, Pointer, Pointer to Structure, various Programs for Array and Pointer. Strings. Introduction to Strings, Definition, Library Functions of Strings.	

Module - 2	No. of hours
	6
Implementing Data Structures with User-Defined Types: Linked Lists, Stacks, and Queues. Introduction to Data Structures, abstract data types, Stack ADT- definition and operations, Queue ADT- definition and operations, Linear list – Singly linked list implementation, insertion, deletion and searching operations on linear list.	

Module - 3	No. of hours
	6
Advanced Linked Lists: Implementation and Operations of Doubly and Circular Linked Circularly linked lists- Operations for Circularly linked lists, Doubly linked list implementation, insertion, deletion and searching operations, applications of linked lists	

Module - 4	No. of hours
	6
Binary Trees: Implementation and Traversal Techniques Trees – Definitions, tree representation, properties of trees, Binary tree, Binary tree representation, binary tree properties, binary tree traversals, binary tree implementation, applications of trees.	

Module - 5	No. of hours
	6
Graph Definition and properties of graphs, Types of graphs, Graph representations, Example of a basic graph implementation, Importance of graph traversal, BFS and DFS on Graph, Advantages and Disadvantages of Graph.	

Course outcomes
a) Understand the basic programming concepts needed to implement data structures.
b) Apply user-defined data types to implement linked lists, stacks, and queues.
c) Implement binary trees and perform different traversal methods like inorder, preorder, and postorder.
d) Analyze graph data structures, perform basic operations, and evaluate traversal techniques like BFS and DFS.

Text Books	
1	Karumanchi, N. (2016). Data structures and algorithms made easy (5th ed.). CareerMonk Publications. ISBN: 978-8193245279
2	Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1982). Data structures and algorithms (1st ed.). Addison-Wesley. ISBN: 978-0201000238

Reference books	
1	Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press. ISBN: 978-0262046305
2	Sedgewick, R., & Wayne, K. (2011, with ongoing support). Algorithms (4th ed.). Addison-Wesley Professional. ISBN: 978-0321573513
3	Siddiqui, A. T., & Siddiqui, S. A. (2023). Data Structure Using C (1st ed.). CRC Press. ISBN: 978-1003453291

Reference links	
1.	https://www.geeksforgeeks.org/data-structures/

Lab Programs		[60 Hrs]
1	Write a program to find the largest and smallest element in an array.	
2	Write a program to find the factorial of a number using recursion. Write a program to find the nth Fibonacci number using recursion.	
3	Write a program to implement the Sudoku game structure and implement searching operations along rows, columns and grids.	
4	Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Implement the following operations: 1. insert_at_beginning(data): Insert a new node with the given data at the beginning of the list. 2. insert_at_end(data): Insert a new node with the given data at the end of the list. 3. delete_node(data): Delete the first node in the list that contains the given data 4. traverse (): Traverse the list and print the data of each node	
5	Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Then, create a class/structure Stack to represent the stack itself. Implement the following operations: 1. push(data): Push a new node with the given data onto the stack. 2. pop(): Remove and return the top node from the stack. If the stack is empty, return None. 3. peek(): Return the data of the top node without removing it. If the stack is empty, return None 4. empty(): Return True if the stack is empty, otherwise return False	
6	Write a program to create a class/structure Node to represent a node in a singly linked list. Each node should have two attributes: data and next. Then, create a class/structure Queue to represent the queue itself. Implement the following operations: 1. enqueue(data): Add a new node with the given data to the end of the queue. 2. front(): Return the data of the front node without removing it. If the queue is empty, return None. 3. is_empty(): Return True if the queue is empty, otherwise return False	
7	Write a program to create a class/structure DoublyNode to represent a node in a doubly linked list. Each node should have three attributes: data, next, and prev. Then, create a class/structure DoublyLinkedList to represent the linked list itself.	

	Implement the following operations: 1. <code>insert_at_beginning(data)</code>: Insert a new node with the given data at the beginning of the list. 2. <code>insert_at_end(data)</code>: Insert a new node with the given data at the end of the list. 3. <code>traverse_forward()</code>: Traverse the list forward and print the data of each node.
8	Write a program to a class/structure <code>CircularNode</code> to represent a node in a circular linked list. Each node should have two attributes: <code>data</code> and <code>next</code>. Then, create a class/structure <code>CircularLinkedList</code> to represent the linked list itself. Implement the following operations: 1. <code>insert_at_beginning(data)</code>: Insert a new node with the given data at the beginning of the list. 2. <code>traverse()</code>: Traverse the list and print the data of each node.
9	Write a program to create a class/structure <code>TreeNode</code> to represent a node in a binary tree. Each node should have three attributes: <code>data</code>, <code>left</code>, and <code>right</code>. Then, create a class/structure <code>BinaryTree</code> to represent the binary tree itself. Implement the following operations: 1. <code>insert(data)</code>: Insert a new node with the given data into the binary tree (assume a simple insertion without balancing). 2. <code>pre_order_traversal()</code>: Perform a pre-order traversal of the tree and print the data of each node.
10	Write a program to create a class <code>TreeNode</code> to represent a node in a binary tree. Each node should have three attributes: <code>data</code>, <code>left</code>, and <code>right</code>. Then, create a class/structure <code>BinaryTree</code> to represent the binary tree itself. include a method/function <code>find_lca(node1, node2)</code> that takes two node values as input and returns the value of their lowest common ancestor. Assume all values in the tree are unique.
11	Write a program to create a class <code>TreeNode</code> to represent a node in a binary tree. Each node should have three attributes: <code>data</code>, <code>left</code>, and <code>right</code>. Then, create a class/structure <code>BinaryTree</code> to represent the binary tree itself, to include a method/function <code>find_grandchildren(node)</code> that takes a node value as input and returns a list of values of all the grandchildren of the given node.
12	Create a class/structure <code>Graph</code> to represent a graph using an adjacency list. Implement the following operations: 1. <code>add_edge(v, w)</code>: Add an edge between vertices <code>v</code> and <code>w</code>.
13	Create a class/structure <code>Graph</code> to represent a graph using an adjacency list, to include a method/function <code>dfs(start_vertex)</code> that performs a Depth-First Search starting from <code>start_vertex</code> and prints the order of traversal.
14	Create a class/structure <code>Graph</code> to represent an undirected graph using an adjacency list. Include a method/function <code>detect_cycle()</code> that detects if there is a cycle in the graph. The method should return <code>True</code> if a cycle is detected, otherwise <code>False</code>.
15	A social network is represented as an unweighted graph where users are vertices and friendships are edges. Implement a class/structure <code>SocialNetwork</code> that supports the following operations:

- | | |
|--|--|
| | <ol style="list-style-type: none">1. <code>add_friendship(user1, user2)</code>: Add a friendship (edge) between user1 and user2.2. <code>degrees_of_separation(user1, user2)</code>: Find the shortest path (in terms of the number of edges) between user1 and user2 using BFS and return the number of edges in the path. |
|--|--|

